

Team4_CI/CD 계획서

1. 프로젝트 개요

FreeBridge는 CRM 사용자를 대상으로 하는 웹 서비스로, 프론트엔드, 백엔드, AI 서비스를 분리하여 운영하고 있다.

사용자는 `http://freebridge.co.kr/` 도메인을 통해 서비스에 접근하며, 전체 서비스는 AWS 기반 인프라 위에서 동작한다.

본 프로젝트는 반복적인 수동 배포 과정에서 발생할 수 있는 휴먼 에러를 줄이고, 서비스 배포 시간을 단축하며, 변경 사항을 안정적으로 운영 환경에 반영하기 위해 Jenkins 기반 CI/CD 파이프라인을 구축하였다.

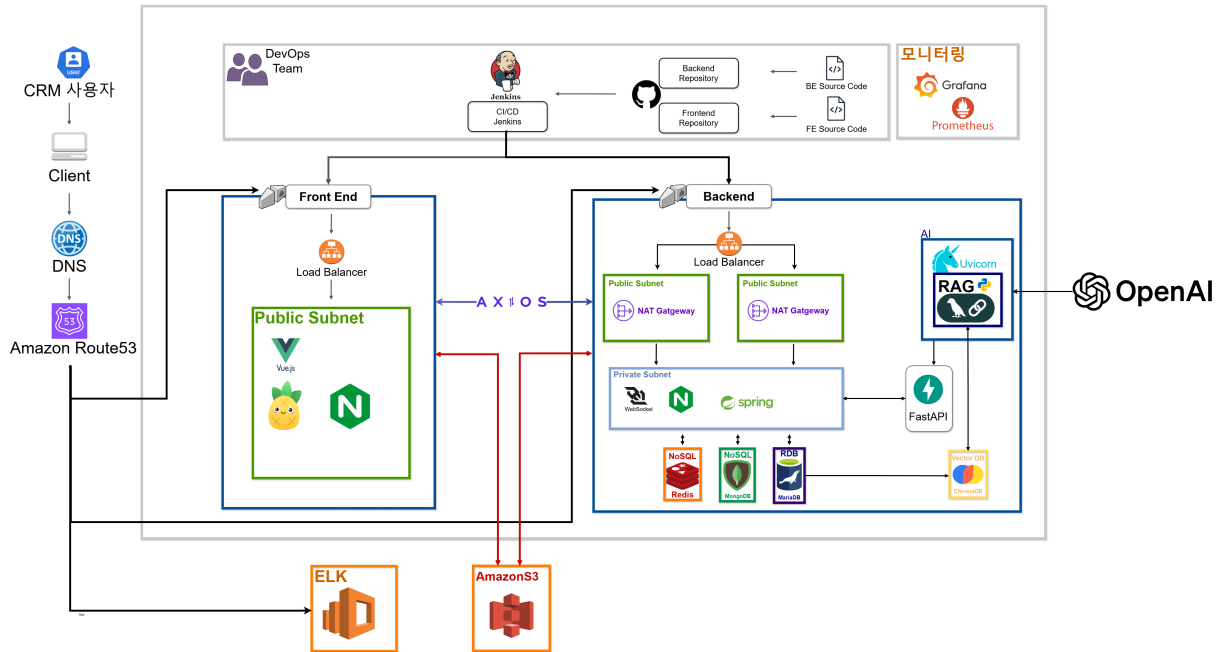
본 계획서에서는 FreeBridge 서비스의 전체 아키텍처와 CI/CD 파이프라인 구조를 설명하고, 프론트엔드, 백엔드, AI 서비스별 자동 배포 흐름과 운영 중인 모니터링 체계를 함께 정리한다.

또한 배포 안정성 확보 방식과 롤백 전략을 제시하여 실제 운영 환경에서의 활용 가능성을 중심으로 서술하고자 한다.

핵심 운영 기준

- 운영 도메인: `http://freebridge.co.kr/`
- 운영 배포 기준 브랜치: `dev`
- 모니터링: `Prometheus + Grafana`
- 롤백 방식: 이전 Docker 이미지 태그로 Manifest 재적용

2. 시스템 아키텍처



FreeBridge의 서비스 아키텍처는 크게 사용자 접근 영역, 애플리케이션 서비스 영역, AI 서비스 영역, 데이터 저장 영역, 그리고 배포 자동화 및 모니터링 영역으로 구성된다.

사용자는 Route53과 DNS를 통해 서비스에 접근하며, 프론트엔드는 Vue.js와 Nginx 기반으로 운영된다.

백엔드는 Spring과 WebSocket 기반으로 구성되어 있으며, AI 서비스는 FastAPI 위에서 LangChain 기반 RAG 체인을 자체 구현하여 분석 및 추천 기능을 제공한다.

배포 자동화는 GitHub Repository와 Jenkins를 중심으로 구성되어 있으며, Jenkins는 프론트엔드와 백엔드 저장소의 변경 사항을 감지해 Docker 이미지를 빌드하고, Docker Hub 및 Manifest Repository를 거쳐 Amazon EKS에 배포를 반영한다.

운영 중인 서비스는 Prometheus와 Grafana를 통해 모니터링하고 있다.

아키텍처 구조도에는 ELK 영역이 함께 표현되어 있으나, 현재 실제 운영 환경에서 적용 중인 모니터링 체계는 Prometheus + Grafana 중심이다.

따라서 본 계획서에서는 현재 구현 및 운영 중인 범위를 기준으로 설명하며, ELK는 향후 확장 가능 항목으로 본다.

3. CI/CD 구축 목적

FreeBridge의 CI/CD 구축 목적은 다음과 같다.

- 개발자가 `dev` 브랜치에 변경 사항을 반영하면 자동으로 빌드와 배포가 수행되도록 하여 배포 속도를 높인다.

- 프론트엔드, 백엔드, AI 서비스를 동일한 배포 체계 안에서 관리하여 운영 일관성을 확보한다.
- Docker 이미지 태그와 Manifest Repository를 기반으로 배포 이력을 추적하고, 장애 발생 시 신속한 롤백이 가능하도록 한다.
- 배포 이후에도 Prometheus와 Grafana를 통해 서비스 상태를 지속적으로 검증하여 운영 안정성을 확보한다.

즉, 본 CI/CD 체계는 단순히 코드를 자동으로 배포하는 수준을 넘어서, 실제 운영 서비스의 안정성까지 고려한 배포 자동화 체계를 목표로 한다.

4. 전체 CI/CD 흐름



FreeBridge의 전체 CI/CD 흐름은 다음과 같다.

1. 개발자가 프론트엔드 또는 백엔드 저장소의 **dev** 브랜치에 코드를 Push한다.
2. GitHub Webhook을 통해 Jenkins Pipeline이 자동 실행된다.
3. 프론트엔드는 Vue/Vite 기반 애플리케이션을 Docker 이미지로 빌드한다.
4. 백엔드는 Gradle을 사용해 Spring Boot 애플리케이션을 빌드한 뒤 Docker 이미지로 패키징한다.
5. AI 서비스 역시 FastAPI 기반 이미지를 별도로 생성한다.
6. 생성된 이미지는 Docker Hub에 업로드된다.

7. Jenkins는 Manifest Repository의 Kubernetes 배포 파일에서 이미지 태그를 최신 버전으로 갱신한다.
 8. Jenkins가 Amazon EKS 클러스터에 `kubectl apply` 를 수행하여 실제 운영 환경에 반영한다.
 9. 배포 완료 후 Prometheus와 Grafana를 통해 주요 운영 지표를 확인한다.
 10. 이상 징후 발생 시 이전 Docker 이미지 태그를 기준으로 Manifest를 재적용하여 롤백한다.
-

5. Frontend CI/CD

5.1 배포 흐름

Frontend CI/CD는 사용자가 접속하는 웹 화면을 최신 상태로 안정적으로 제공하기 위한 파이프라인이다.

개발자가 Frontend Repository의 `dev` 브랜치에 변경 사항을 Push하면 Jenkins가 자동으로 파이프라인을 실행한다.

이후 Docker 빌드 과정에서 Vue/Vite 산출물을 생성하고, 이를 Nginx 기반 이미지에 포함시켜 Docker Hub에 업로드한다.

Jenkins는 업로드된 이미지 태그를 Manifest Repository의 `frontend-deployment.yml` 에 반영하고, EKS에 Deployment, Service, Ingress를 적용하여 실제 서비스 환경을 갱신한다.

배포가 완료되면 사용자는 `http://freebridge.co.kr/` 도메인을 통해 최신 프론트엔드 화면에 접근할 수 있다.

프론트엔드는 Replica 2개를 유지한 상태에서 RollingUpdate 전략으로 반영되므로, 서비스 중단 없이 새 버전으로 전환할 수 있다.

5.2 배포 전략

- Jenkins 기반 자동 배포
- Docker Hub를 통한 이미지 버전 관리
- Kubernetes `RollingUpdate` 기반 무중단 배포
- Ingress를 통한 `/` 경로 라우팅
- 이전 이미지 태그 기준 롤백 가능

▼ Frontend 핵심 파이프라인 코드 보기

프론트엔드 빌드 및 업로드 핵심 코드

```
docker build \
  --build-arg VITE_API_BASE_URL="${FRONTEND_API_BASE_URL:-}" \
  -t ${IMAGE_NAME}:${IMAGE_TAG} .

docker push ${IMAGE_NAME}:${IMAGE_TAG}
docker tag ${IMAGE_NAME}:${IMAGE_TAG} ${IMAGE_NAME}:latest
docker push ${IMAGE_NAME}:latest
```

위 코드는 프론트엔드 소스 코드를 Docker 이미지로 패키징하고, 버전 태그 이미지와 `latest` 이미지를 함께 Docker Hub에 업로드하는 단계이다.

Manifest 반영 핵심 코드

```
sed -i 's|image: ${env.IMAGE_NAME}:.+|image: ${env.IMAGE_NAME}:${env.IMAGE_TAG}|g' kube-folder/frontend-deployment.yml
kubectl apply -f kube-folder/frontend-deployment.yml
kubectl apply -f kube-folder/frontend-service.yml
kubectl apply -f kube-folder/ingress-set.yml
```

위 단계는 프론트엔드 이미지 태그를 최신 빌드 버전으로 변경한 뒤, EKS에 실제 배포 설정을 반영하는 역할을 한다.

6. Backend CI/CD

6.1 배포 흐름

Backend CI/CD는 Spring 기반 API 서버와 WebSocket 기능을 안정적으로 운영 환경에 반영하기 위한 파이프라인이다.

개발자가 Backend Repository의 `dev` 브랜치에 변경 사항을 Push하면 Jenkins가 이를 감지하고 빌드를 시작한다.

Jenkins는 먼저 Gradle을 이용해 Spring Boot 애플리케이션을 JAR로 빌드한 뒤, 이를 Docker 이미지로 패키징하여 Docker Hub에 업로드한다.

이후 Manifest Repository의 `backend-deployment.yml`에 최신 이미지 태그를 반영하고, EKS에 Secret, Deployment, Service를 적용한다.

Backend Deployment는 Replica 2개로 구성되어 있으며, `livenessProbe`와 `readinessProbe`를 통해 새로 생성된 Pod가 정상 상태에 도달한 뒤에만 실제 트래픽을 받도록 설계되어 있다.

따라서 서비스 중단 없이 새로운 백엔드 버전을 배포할 수 있다.

6.2 배포 전략

- Jenkins 기반 자동 배포
- Gradle BootJar 빌드 후 Docker 이미지 생성
- Kubernetes `RollingUpdate` 기반 무중단 배포
- Readiness/Liveness Probe 기반 안정성 검증
- `/api`, `/ws/chat` 경로를 Backend Service로 라우팅
- 이미지 태그 이력을 통한 롤백 지원

▼ Backend 핵심 파이프라인 코드 보기

빌드 및 버전 태그 생성 핵심 코드

```
sh 'chmod +x freebridge/gradlew'
sh 'cd freebridge && ./gradlew clean bootJar -x test'

env.GIT_COMMIT_HASH = sh(script: 'git rev-parse --short HEAD', returnStdout: true).trim()
env.IMAGE_TAG = "${currentBuild.number}-${env.GIT_COMMIT_HASH}"
```

위 코드는 백엔드 애플리케이션을 JAR 형태로 빌드하고, Jenkins 빌드 번호와 Git Commit Hash를 조합해 배포 추적이 가능한 이미지 태그를 생성하는 단계이다.

배포 반영과 검증 핵심 코드

```
sh "DOCKER_BUILDKIT=${env.DOCKER_BUILDKIT} docker build --build-arg APP_JAR=freebridge/app-main/build/libs/app-main-0.0.1-SNAPSHOT.jar -t${env.IMAGE_NAME}:${env.IMAGE_TAG} ."
sed -i "s|image:${env.IMAGE_NAME}:.*|image:${env.IMAGE_N
```

```
AME}:${env.IMAGE_TAG}|g" kube-folder/backend-deployment.
yml
kubectl apply -f kube-folder/backend-deployment.yml
kubectl rollout status deployment/backend --timeout=600s
```

이 코드는 JAR 파일을 기반으로 Docker 이미지를 생성한 뒤, Manifest Repository의 배포 파일을 최신 이미지 태그로 갱신하고 EKS에 반영하는 역할을 한다.

7. AI 서비스 CI/CD

7.1 배포 흐름

AI 서비스는 FastAPI 기반으로 구성되어 있으며, LangChain 기반 체인을 활용한 RAG 기능을 자체 구현하여 백엔드와 연동된다.

현재 AI 서비스는 백엔드 파이프라인과 함께 통합적으로 배포된다.

Backend Repository의 변경 사항이 반영되면 Jenkins가 백엔드와 함께 AI 서비스 이미지를 빌드하고, 이를 Docker Hub에 업로드한다.

그 후 `python-ai-deployment.yml`에 최신 이미지 태그를 반영하고, ConfigMap과 Secret을 포함한 Kubernetes 리소스를 EKS에 적용한다.

AI 서비스는 내부 네트워크를 통해 Backend와 통신하며, ChromaDB 및 데이터베이스 정보와 연계하여 분석 및 추천 기능을 수행한다.

7.2 배포 전략

- Backend 파이프라인과 통합 운영
- FastAPI 기반 AI 서비스 별도 이미지 생성
- ConfigMap, Secret 기반 설정 분리
- Kubernetes `RollingUpdate` 기반 배포
- Backend 내부 호출 구조를 통한 AI 연동

▼ AI 서비스 핵심 파이프라인 코드 보기

```
sh "cd freebridge-ai && DOCKER_BUILDKIT=${env.DOCKER_BUI
LDKIT} docker build -t${env.AI_IMAGE_NAME}:${env.IMAGE_T
AG} ."
sed -i "s|image:${env.AI_IMAGE_NAME}:.*|image:${env.AI_I
MAGE_NAME}:${env.IMAGE_TAG}|g" kube-folder/python-ai-dep
```

```

loyment.yml
kubectl apply -f kube-folder/python-ai-configmap.yml
kubectl apply -f kube-folder/python-ai-secret.yml
kubectl apply -f kube-folder/python-ai-deployment.yml
kubectl rollout status deployment/python-ai-service --ti
meout=600s

```

위 코드는 AI 서비스를 별도 이미지로 빌드하고, 운영 설정을 포함한 Kubernetes 리소스를 함께 반영하는 단계이다.

8. 운영 및 모니터링 전략



FreeBridge는 배포 이후 운영 상태를 지속적으로 검증하기 위해 Prometheus와 Grafana 기반 모니터링 체계를 운영하고 있다.

Prometheus는 백엔드 애플리케이션의 메트릭을 수집하며, Grafana는 이를 시각화하여 운영자가 서비스 상태를 실시간으로 확인할 수 있도록 한다.

현재 Backend 서비스에 대해 ServiceMonitor를 구성하여 주요 메트릭을 주기적으로 수집하고 있다.

첨부한 Grafana 대시보드에서는 Backend Up, 5xx Error Rate, Traffic RPS, P95 Latency, JVM Heap, Tomcat Threads, HikariCP, CPU Usage 와 같은 핵심 지표를 확인할 수 있다.

예시 화면에서 Backend Up 값이 2로 표시되는 것은 두 개의 백엔드 Replica가 정상적으로 동작하고 있음을 의미하며, 5xx Error Rate 가 0 req/s로 유지되는 것은 현재 서버 오류가 발생하지 않고 있음을 보여준다.

또한 `P95 Latency` 와 `Traffic RPS` 를 통해 사용자 요청 처리 성능을 확인할 수 있고, `JVM Heap` , `Tomcat Threads` , `HikariCP` , `CPU Usage` 를 통해 애플리케이션 내부 자원 상태를 점검할 수 있다.

이와 같은 모니터링 체계는 배포 성공 여부를 단순히 Jenkins 결과로만 판단하지 않고, 실제 운영 중인 서비스 상태까지 함께 검증할 수 있게 해준다.

즉, FreeBridge의 CI/CD는 배포 자동화와 운영 모니터링이 결합된 형태로 구성되어 있다.

9. 배포 안정성 및 롤백 전략

FreeBridge는 실제 운영 환경에서 안정적인 배포를 보장하기 위해 사전 점검, 무중단 배포, 상태 검증, 알림, 롤백 절차를 함께 구성하였다.

- Jenkins Pipeline은 배포 전에 `aws` , `kubectl` , `cluster-info` , `node` , `namespace` 상태를 확인하는 EKS Preflight 단계를 수행한다.
- 프론트엔드와 백엔드는 RollingUpdate 방식으로 배포된다.
- 백엔드는 Liveness/Readiness Probe를 통해 비정상 Pod가 트래픽을 받지 않도록 설정하였다.
- Jenkins는 배포 성공 및 실패 결과를 Discord 알림으로 전달한다.

롤백 방식

- Docker Hub에 저장된 이전 이미지 태그를 기준으로 Manifest Repository의 배포 파일을 이전 버전으로 되돌린다.
- 이후 해당 Manifest를 EKS에 재적용하여 안정 버전으로 복구한다.

이 방식은 Jenkins 빌드 번호와 Git Commit Hash를 조합한 이미지 태그를 사용하기 때문에 특정 시점의 배포 버전을 명확하게 식별할 수 있으며, 빠른 복구가 가능하다는 장점이 있다.

10. 향후 고도화 계획

현재 FreeBridge의 CI/CD 파이프라인은 빌드, 이미지 생성, 이미지 업로드, Manifest 갱신, Kubernetes 배포 반영까지의 흐름을 자동화하고 있다.

그러나 CI 단계에서의 테스트 품질 게이트는 아직 충분히 강화되지 않은 상태이다.

실제 백엔드 파이프라인은 `bootJar -x test` 형태로 테스트를 생략하고 있으며, 프론트엔드 역시 이미지 빌드와 배포 중심으로 구성되어 있다.

따라서 향후에는 다음과 같은 항목을 추가하여 CI 신뢰도를 더욱 높일 계획이다.

- 단위 테스트 자동화

- 정적 분석
 - 보안 취약점 점검
 - 배포 승인 절차
 - 로그 기반 분석 체계 확장
-

11. 결론

본 프로젝트는 Jenkins, Docker Hub, Manifest Repository, Amazon EKS를 중심으로 프론트엔드, 백엔드, AI 서비스를 통합적으로 배포하는 CI/CD 체계를 구축하였다.

개발자가 `dev` 브랜치에 코드를 반영하면 자동으로 빌드와 배포가 수행되고, 운영 환경에는 무중단 방식으로 변경 사항이 적용된다.

또한 Prometheus와 Grafana를 통한 운영 모니터링, 이미지 태그 기반 롤백 전략을 함께 구성함으로써 단순 자동 배포를 넘어 실제 운영 안정성을 고려한 CI/CD 체계를 구현하였다.

이를 통해 배포 시간을 단축하고, 수동 작업으로 인한 오류를 줄이며, 서비스 장애 발생 시 빠르게 복구할 수 있는 기반을 마련하였다.

FreeBridge의 CI/CD 체계는 향후 테스트 자동화와 품질 검증 단계를 추가함으로써 더욱 완성도 높은 운영형 배포 시스템으로 발전할 수 있을 것으로 기대한다.